



Privacy-preserving anomaly detection in cloud with lightweight homomorphic encryption

Abdulatif Alabdulatif*, Heshan Kumarage, Ibrahim Khalil, Xun Yi

Computer Science & Software Engineering, School of Science, RMIT university, Australia

ARTICLE INFO

Article history:

Received 21 December 2015

Received in revised form 14 September 2016

Accepted 1 March 2017

Available online 20 March 2017

Keywords:

Data privacy

Anomaly detection

Data clustering

Homomorphic Encryption

Cloud computing

ABSTRACT

Anomaly detection on large-scale, complex and dynamic data is an essential service that is vital to enable smart functionality in most systems. Increased reliance on cloud computing infrastructures to process such data pose critical challenges with regard to security and privacy. This paper introduces a practical framework that takes advantage of cloud resources to provide a lightweight and scalable privacy preserving anomaly detection service for sensor data. A lightweight Homomorphic Encryption scheme is used to ensure data security and privacy with any computational limitations overcome through a convenient data processing model that employs a single private server collaborating with a set of public servers within a cloud data centre. Virtual nodes implemented on public servers perform granular anomaly detection operations on encrypted data. Comprehensive experimentation demonstrates consistently high detection accuracy with less overheads in a cloud-based anomaly detection model that is both lightweight and scalable while ensuring data privacy.

© 2017 Elsevier Inc. All rights reserved.

1. Introduction

In the current networked landscape, smart functionality over various services is enabled through the implementation of continuous, ubiquitous sensing and the analysis of gathered data. A key application in this context is to ensure the integrity of the data that may be compromised due to faulty sensors, malware corruption or various security attacks through an effective anomaly detection procedure. This is an increasingly vital task to be performed prior to the various analysis techniques that work to extract knowledge and enable decision support in different applications [1]. The exponential growth in the amounts of data collected on these applications lead to significant challenges that relate to the underlying large-scale, heterogeneity and complexity of data while providing efficient and scalable anomaly detection services [2]. Essentially, the recent big data paradigm in these application areas with variety, volume and veracity creates significant challenges in terms of the computational and storage capabilities with regard to providing timely analysis for decision support [3]. These challenges are increasingly overcome by adapting cloud computing technology to provide a convenient framework in implementing services like anomaly detection on such applications [4]. Therefore, large-scale data processing is moving towards a cloud computing paradigm that has access to significant computational capabilities with massive storage capacity and supports distributed on-demand access [5]. These cloud features can be exploited in a manner that leads to improvements

* Corresponding author at: 124 Little La Trobe St., Melbourne, Vic 3000, Australia.

E-mail addresses: Abdulatif.alabdulatif@rmit.edu.au (A. Alabdulatif), heshandkr@gmail.com (H. Kumarage), Ibrahim.khalil@rmit.edu.au (I. Khalil), xun.yi@rmit.edu.au (X. Yi).

<http://dx.doi.org/10.1016/j.jcss.2017.03.001>

0022-0000/© 2017 Elsevier Inc. All rights reserved.

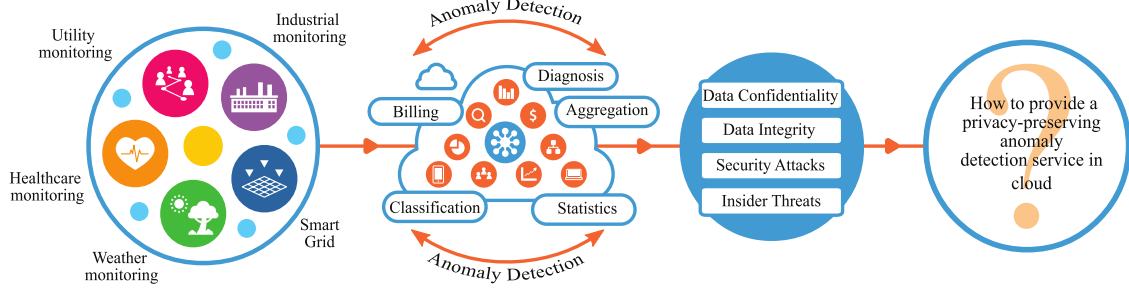


Fig. 1. A variety of applications that generate large-scale data require anomaly detection to enable smart functionality and decision support. The large volumes of data and the need for efficient data processing facilities mandates the use of cloud infrastructure which introduces critical problems of data ownership, security and privacy.

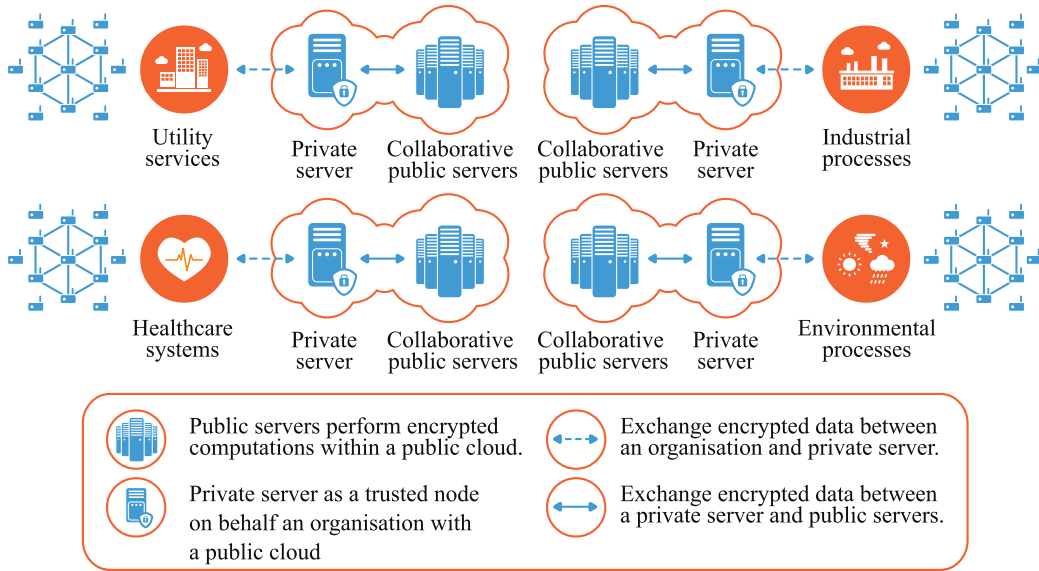


Fig. 2. An overview of the cloud computing architecture proposed for privacy-preserving anomaly detection as a service in cloud. Organisations can delegate specific anomaly detection tasks to the cloud where public servers collaborate with a private server to perform required sub-tasks in a secure manner with homomorphic encryption.

in performance and efficiency of anomaly detection services when compared to traditional data processing approaches for anomaly detection, which are mostly non-scalable given the current landscape of big data analytics.

However, despite the benefits of adapting a cloud computing paradigm for enabling efficient anomaly detection services, significant challenges in data security and privacy have to be overcome [6]. Most applications, ranging from industrial and machine monitoring to personal healthcare systems, deal with private data sets that cannot be exposed to any third party due to individual sensitivity or the risk of compromising corporate privacy. A key concern for owners of such data is whether they have control over their data irrespective of where it is located in the data processing infrastructure [7]. In this regard, various solutions have been proposed based on different perspectives such as cryptography and third party participation [8] to ensure the security and privacy of cloud-based data processing. These facts are illustrated in a summarised manner in Fig. 1. The focus of this paper is therefore, to provide an effective cryptographic data processing model to preserve data privacy in performing anomaly detection as a cloud-based service for large-scale and distributed data applications. We investigate the viability of adopting an efficient *Lightweight Homomorphic Encryption* technique that supports mathematical computations on encrypted data for such a distributed cloud-based anomaly detection framework.

More specifically, we propose a secure cloud-based data processing solution for large-scale sensor data anomaly detection that is both scalable and adaptive for large amounts of dynamic data streams while preserving individual data privacy. Sensor data is first encrypted using a lightweight Homomorphic Encryption (HE) scheme before being communicated to the cloud infrastructure for analysis while keeping the network topology information of the original network intact. Fig. 2 provides an overview of the proposed architecture consisting of the particular cloud infrastructure and the application. A key feature of this architecture is that each data centre that handles a certain organisation's data consists of a single private server owned by that organisation. This particular server will handle some limited operations that are not supported by the adapted HE scheme with regard to implementation of the anomaly detection service. All other operations on the

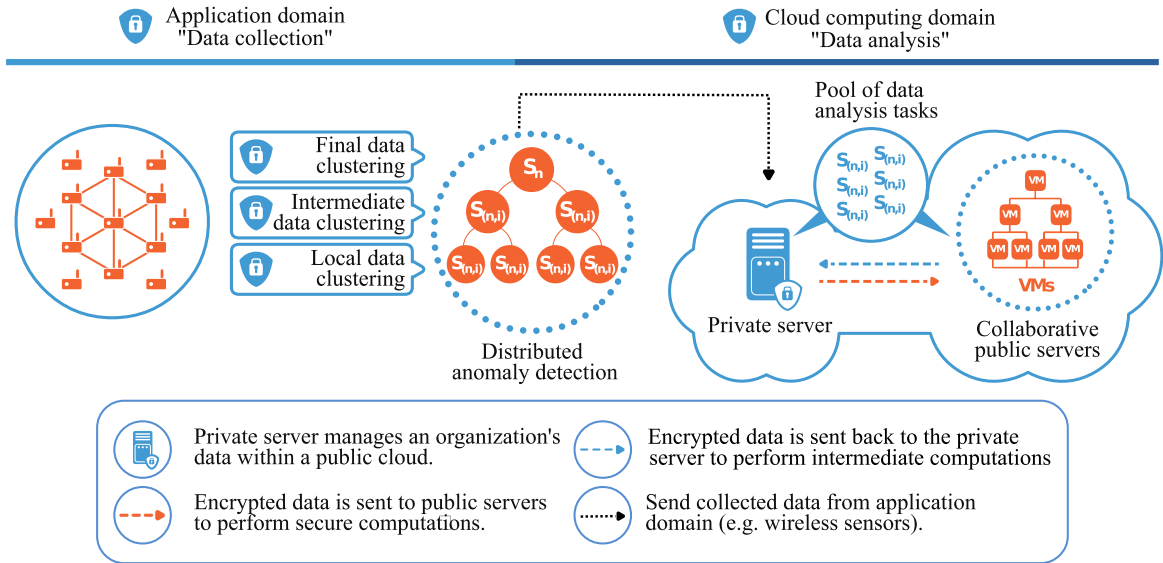


Fig. 3. Hierarchical distributed data processing architecture for lightweight and privacy preserving anomaly detection. The hierarchical clustering process to facilitate anomaly detection (based on Kumarage et al. [9]) is moved to the cloud in a compatible manner where the data processing sensor nodes are mapped to hierarchical structure of virtual machines (VMs). In the proposed model, both private and collaborative public cloud servers collaborate to manage and perform clustering tasks securely and a privacy-preserving manner while located in the same data centre.

encrypted data will be performed by the public servers in the cloud data centre. As the private server is assumed to be in the same data centre there are no unsustainable communication overheads. The different data analytics tasks in the proposed anomaly detection algorithm are mapped onto separate virtual machines on the public cloud in a manner that reflects the network architecture of the source sensor network application. Tasks that require a computation task beyond the capability of the lightweight encryption scheme employed is performed on the private server.

The specific anomaly detection technique used in the proposed framework is based on the work presented in [9] and uses a distributed and fuzzy data clustering scheme to efficiently partition the data and classify anomalies. The distributed data clustering procedure is therefore adopted in the proposed framework over the introduced cloud computing architecture as depicted in Fig. 2. Different phases of the data clustering and anomaly detection scheme over a distributed hierarchical topology is to be processed through virtual machines (VMs) that are dynamically managed by a VM management process. Fig. 3 illustrates this mapping process with the cluster analysis tasks performed hierarchically over the cloud computing architecture by leveraging different VM entities in communication with the private server. Overall the proposed framework works to further increase the efficiency of the original cluster-analysis process for anomaly detection proposed in [9] by leveraging on the cloud infrastructure.

1.1. Contribution

The proposed model in this paper solves the key issues of providing a scalable anomaly detection service for large-scale heterogeneous data by leveraging on cloud resources through a secure and privacy-preserving data processing approach. The privacy of the data in the proposed model is ensured by utilising an appropriate HE technique prior to processing the data. A major concern in this context is the efficiency of the underlying homomorphic scheme. The specific contributions of the paper can be highlighted as follows.

- A cloud-based data processing model is presented for performing anomaly detection in a privacy-preserving manner for large-scale sensor data. Data processing is performed in a public cloud data centre complemented by a single server that is owned privately by the organisation that requires the analytics service and owns the data. lightweight HE is applied on all sensed data prior to analysis on the cloud. However, due to limitations in current HE methodologies some of the required operations in the overall anomaly detection scheme (e.g. division operations) have to be done on decrypted data. These operations are done on the trusted private server while all other operations are performed in the public cloud infrastructure on the encrypted data itself. The communication between these two entities is encrypted to ensure end-end privacy and there are very few communication costs as the private server is located in the public data centre itself. It is shown that this model is capable of performing anomaly detection as a cloud-based service both accurately and efficiently through a distribution of different analytical tasks in the cloud infrastructure. A lightweight HE scheme that is able to carry computations on data while it is encrypted, is used to protect data privacy in the cloud computing infrastructure. Specifically, Domingo-Ferrer's additive and multiplicative privacy homomorphism scheme [10] is employed due to its relative simplicity in implementation as well as scalability aspects to encrypt all data prior to

communication and processing. Domingo-Ferrer's scheme is shown to be particularly successful in maintaining trade-off between performance efficiency and computational capabilities when compared to other potential schemes.

- In addition, we provide a detailed description of the different secure operations and their implementation in the proposed hybrid data processing model between a private server and the public cloud infrastructure. In the evaluation section, results of comprehensive experimentation demonstrate that the proposed framework can preserve data privacy without affecting the accuracy of the anomaly detection process. Comparisons for detection accuracy between plaintext and ciphertext are presented to be comparable with our previous work [9] where the same anomaly detection model was applied on non-encrypted data on a more resource constrained non-cloud environment. Experiment results of privacy-preserving anomaly detection show an average sensitivity of 95% and an average specificity of 99% with a high processing efficiency.

The rest of the paper is organised as follows. A detailed discussion on related work is presented in Section 2. The model architecture and its specifications with the security analysis are then presented in Section 3. The proposed model is evaluated in Section 4 by conducting experiments on real-world data sets in our cloud-based model. We conclude the paper in Section 5.

2. Related work

We briefly review recent research on privacy-preserving data mining on cloud environments and the applicability of a Homomorphic Encryption (HE) paradigm for such tasks in this section.

Secure computation allows single or multiple parties to execute specific functions on data inputs and return results without compromising data privacy. In practice, several applications have single or multiparty computations, especially those that perform statistical analysis computations shared by distributed multiparties. The main concern with such applications is how to preserve data privacy while retaining the desired analytic functionality. In this paper, we focus on cryptographic approaches that are used to secure cloud-based computations for massive amounts of data. Although several cryptographic techniques can be adapted to secure cloud-based computations, we focus our discussion on the following approaches that can be considered as candidates to achieve privacy preserving data computation in a scalable and lightweight manner [11]; Secure multiparty computation (SMC), verifiable computation (VC) and homomorphic encryption (HE).

SMC was introduced by Yao [12] and extended to multiparties by Chaum, Crépeau, and Damgård [13]. This is considered an important part of cryptography concepts and is directly involved in data analysis applications. In general, there are two common techniques for multiparty computations: garbled circuits, such as [14], and secret sharing, such as [13] and [15]. These techniques are involved in many existing multiparty computation protocols in cloud computing. However, SMC techniques are either exposed to several security vulnerabilities from shared parties (e.g. Trusted Third Party) which may be corrupted by an adversary [11] or have high communication overheads. Especially in case of large-scale data, extensive communications occur between parties or within TTP to perform computations in a secure manner.

Verifiable computation (VC), as in Pinocchio [16], allows the data owners to check the integrity of performed computations. In the VC scheme, data owner gives the data, along with the requirements of the computation desired, to a powerful entity, called the prover. The prover then outputs the result of the required computation, along with a proof of the output correctness. The VC is mainly used to ensure data integrity while it is delegated to a cloud-based provider to perform specific computations. However, it does not support data confidentiality compared with both SMC and HE approaches, which is an essential requirement when data computation is delegated to an untrusted cloud-based provider.

Homomorphic encryption (HE) On other hand, has the ability to perform computations directly on encrypted data as if it is performed on plaintext data. We believe that HE can be an ideal and convenient approach compared to others, as it does not require interaction between parties to perform computations and avoids the need to have a TTP which may lead to several security vulnerabilities. HE has existed since the inception of public key cryptography. Examples of HE include RSA [17] ElGamal [18] Benaloh [19] and Paillier [20]. However, these are somewhat homomorphic encryption (SHE) schemes, which can only support a limited number of operations (either addition or multiplication, not both). A number of these schemes fail to be appropriate for practical applications because they are often inefficient and are unable to perform a variety of arithmetic computations that are required to build useful applications in cloud.

In contrast a fully homomorphic encryption (FHE) scheme would support a variety of arithmetic computations simultaneously. This was a major goal that remained unattained until the breakthrough result of Gentry [21], which is based on the properties of ideal lattices [19]. Shortly after, several FHE schemes were developed, such as Brakersk et al. [22] and other [23–25]. While these schemes are proposed to reduce the complexity of an original FHE scheme [21], they are still impractical for use in large-scale applications due to a common drawback: they include computational overheads in terms of efficiency and execution time that are impractical over current cloud applications. This led to the conclusion that FHE is not efficient enough for practical applications, especially in a cloud paradigm. Table 1, shows some selected HE techniques and their capabilities. A comprehensive review of HE techniques is given in [26].

Considering the limitations discussed above, we adapt a lightweight and provably secure additive and multiplicative privacy homomorphism scheme [30] which has the ability to securely perform main basic arithmetic operations (addition, subtraction and multiplication) on encrypted data. It is a symmetric-key encryption scheme that is secure against Chosen-Ciphertext Attacks (CCA) [31]. Domingo-Ferrer's scheme is characterized by lightweight and efficient computational

Table 1
Homomorphic cryptosystems.

Cryptosystem	Operation capabilities			Key type	Security aspect
	Addition	Subtraction	Multiplication		
Paillier [20]	X			Asymmetric	Semantically secure
El-Gamal [18]			X	Asymmetric	Semantically secure
Okamoto–Uchiyama [27]	X			Asymmetric	Provably secure
Naccache–Stern [28]	X			Asymmetric	Semantically secure
RSA [17]			X	Asymmetric	Not semantically secure
Elptic curve El-Gamal [29]	X			Asymmetric	Elliptic curve discrete logarithm problem
Domingo-Ferrer [30]	X	X	X	Symmetric	Secure against chosen ciphertext attack

capabilities when compared with other HE schemes. Also, it can perform an unlimited variety of arithmetic operations securely based on homomorphic properties. These features make Domingo-Ferrer's scheme a convenient solution to preserving the privacy of cloud applications. We believe that the Domingo-Ferrer's additive and multiplicative privacy homomorphism scheme [30] is one of the most applicable HE schemes for data analysis purposes. Indeed, it has a convenient encryption/decryption mechanism, which helps to efficiently implement different tasks and operations on various cloud-based applications. A brief but detailed description of Domingo-Ferrer's scheme is included in section 3.

3. Lightweight privacy-preserving model for anomaly detection in cloud

In the section, we propose a scalable and privacy preserving model for cloud based anomaly detection using a lightweight homomorphic approach. Our model is implemented to perform distributed data clustering for anomaly detection on encrypted data by leveraging on the reliability and scalability of cloud computing resources. However, an anomaly detection scheme, as in any other data analysis technique, involves various mathematical computations; some of which cannot be carried out completely on an encrypted domain based on existing homomorphic schemes with tolerable levels of computational complexity. We overcome this issue by adapting the privacy manager concept [32] through a secure privately-owned server entity collaboration within a public cloud data centre.

The proposed model can therefore perform secure computations that are required (addition, subtraction, multiplication, division and comparison) by using HE within the public cloud computing infrastructure, complemented by the private server (the privacy manager). The computations to be performed in this server are those that need decryption due to limitations in the adapted HE scheme. However, as it is assumed that this server belongs to the client organisation as a trusted entity while located in the same public cloud data centre where the analytics service is carried out, the communication overheads that occur in such situations where data need to be communicated to this server are mitigated.

In detail, the proposed model has two main entities. The first is the private server that is assumed to be located in the public cloud infrastructure. It is responsible for managing both plaintext and ciphertext data and also encryption/decryption mechanisms which must be performed in a secure manner with regard to the specific operations that cannot be performed without decryption in the public cloud infrastructure. The second entity is the collaborative servers with individual virtual machines (VMs) that are considered to be public servers within a public cloud computing environment and are responsible for performing data analysis computations that can be performed on ciphertext. These two entities collaborate in the proposed model to perform scalable anomaly detection on data that is encrypted using a HE scheme. We look at the specific sub tasks of homomorphic encryption and data clustering and anomaly detection in the next sections. These are followed by sections describing the overall system workflow and a security analysis discussion.

3.1. Lightweight Homomorphic Encryption

We build our model based on HE that can ensure end-to-end security. Domingo-Ferrer's additive and multiplicative privacy homomorphism scheme [30] is considered a lightweight HE scheme in this aspect. We illustrate Domingo-Ferrer's scheme specification in detail below.

The Domingo-Ferrer's scheme has two groups of parameters: **public parameters** (m, v), and **private parameters** ($\hat{m}, \hat{r}$).

- The two public parameters are represented by a large integer with many divisors (m) and a small integer that determines a vector space (v), which is a corresponding ciphertext of a plaintext value.
- The two secret parameters are represented by a small divisor of m , ($\hat{m}$) where $\hat{m} > 1$ and ($\hat{r}$) $\in Zm$, where r^{-1} modulus m must exist.

There are two levels of operations where the classified level is a secure entity that perform initial encryption operations and final decryption operations; the unclassified level is in a public environment and can perform mathematical computation on encrypted data (e.g. public cloud computing).

Algorithm 1 Encryption and decryption operations.

```

1: procedure (1)(Encryption operation)
2:   Inputs: a plaintext value ( $\mathbf{a}$ ),  $m$ ,  $\hat{m}$ ,  $v$ ,  $\hat{r}$ 
3:   Outputs:  $\langle E(a_1), E(a_2), \dots, E(a_v) \rangle \leftarrow$  ciphertext as a vector
4:   Initialisation:  $r^{-1} \bmod m$  exists,  $(0, \dots, 0) \leftarrow c$  an empty vector
5:    $c = (a_1, a_2, \dots, a_v)$  generate  $v$  random values based on Equation (1)
6:    $c = E(a) = (a_1 r \bmod m, a_2 r^2 \bmod m, \dots, a_v r^v \bmod m)$ 
7:    $c = \langle E(a_1), E(a_2), \dots, E(a_v) \rangle$ 
8: end procedure
9: -----
10: procedure (2)(Decryption operation)
11:   Inputs:  $\langle E(a_1), E(a_2), \dots, E(a_v) \rangle$ ,  $\hat{m}$ ,  $m$ ,  $r - 1$ ,  $sum = 0$ ;
12:   Outputs: a plaintext value ( $\mathbf{a}$ )
13:    $a = (E(a_1) \times r^{-1} \bmod m, E(a_2) \times r^{-2} \bmod m, \dots, E(a_v) \times r^{-v} \bmod m)$ 
14:    $a = \langle E(a_1), E(a_2), \dots, E(a_v) \rangle$ 
15:   for ( $i = 1, i \leq v, i++$ ) do
16:      $sum = sum + E(a_i)$ 
17:   end
18:    $a = sum \bmod \hat{m}$ 
19: end procedure

```

Algorithm 2 Addition and subtraction operations.

```

1: procedure
2:   Inputs:  $c_1 = \langle E(a_1), E(a_2), \dots, E(a_v) \rangle$ ,  $c_2 = \langle E(b_1), E(b_2), \dots, E(b_v) \rangle$ 
3:   Outputs:  $\langle E(r_1), E(r_2), \dots, E(r_v) \rangle \leftarrow$  ciphertext as a vector
4:   Initialisation:  $result = \langle E(r_1), \dots, E(r_v) \rangle$ 
5:    $c_1 \pm c_2 = ((E(a_1) \pm E(b_1) \bmod m), (E(a_2) \pm E(b_2) \bmod m), \dots, (E(a_v) \pm E(b_v) \bmod m))$ 
6:    $result = \langle E(r_1), E(r_2), \dots, E(r_v) \rangle$ 
7: end procedure

```

- **DF.Encryption** ($a, m, \hat{m}, \hat{r}$): The encryption operation has two stages. **Transformation stage** where a plaintext value $\mathbf{a} \in \mathbf{Z}\hat{\mathbf{m}}$ is split into a set of random values $(a_1, a_2, \dots, a_v)$, where v is a number of elements in a vector for each transformed plaintext value. **The encryption stage** is where vector elements are encrypted individually. A generated vector for each plaintext value has to satisfy the following constraint:

$$a = \sum_{i=1}^v a_i \bmod \hat{m} \quad \text{where } a_i \in \mathbf{Z}m \quad (1)$$

We then perform an encryption operation for the vector elements as follows.

$$E(a) = (a_1 \hat{r}^1 \bmod m, a_2 \hat{r}^2 \bmod m, \dots, a_v \hat{r}^v \bmod m) \quad (2)$$

- **DF.Decryption** ($E(a), m, \hat{m}, \hat{r}^{-1}$): The decryption operation is performed by computing a scalar product of the i th element of a vector by $r^{-1} \bmod m$ to find $a_i \bmod m$ as follows:

$$(a_1 r^{-1} \bmod m, a_2 r^{-2} \bmod m, \dots, a_d r^{-d} \bmod m) \quad (3)$$

Next, we aggregate vector elements and, recalling Equation (1), retrieve a plaintext value of $E(a)$ as follows:

$$a = \sum_{i=1}^v a_i \bmod m' \quad (4)$$

Algorithm 1 illustrates the encryption and decryption operations in more detail. The arithmetic computations on encrypted domain are carried out over $(\mathbf{Z}m)^v$ in a public cloud computing environment (an unclassified level).

- **DF.Add** ($E(a), E(b), m$), **DF.Sub** ($E(a), E(b), m$): The addition and subtraction operations between two encrypted ciphertext can be done componentwise between terms (vector elements) with the same degree. For example, we can add or subtract two ciphertexts in case of vector space $v = 2$ as follows.

$$E(a) = (a_1, a_2) \quad , \quad E(b) = (b_1, b_2)$$

$$E(a) \pm E(b) = ((a_1 \pm b_1) \bmod m \quad , \quad (a_2 \pm b_2) \bmod m) \quad (5)$$

(See Algorithm 2.)

- **DF.Mult** ($E(a), E(b), m$): The multiplication operation works as polynomials: all elements are cross-multiplications in $\mathbf{Z}_m$. For example, we can add or subtract two ciphertexts in the case of vector space $v = 2$ as follows.

Algorithm 3 Multiplication operation.

```

1: procedure
2:   Inputs:  $c_1 = \langle E(a_1), E(a_2), \dots, E(a_v) \rangle$ ,  $c_2 = \langle E(b_1), E(b_2), \dots, E(b_v) \rangle$ 
3:   Outputs:  $\langle E(r_1), E(r_2), \dots, E(r_v) \rangle \leftarrow$  ciphertext as a vector
4:   Initialisation: result =  $\langle E(r_1), \dots, E(r_v) \rangle$ 
5:   //Multiplication works as polynomials
6:   // Example of two ciphertext:  $c_1 = \langle E(a_1), E(a_2) \rangle$ ,  $c_2 = \langle E(b_1), E(b_2) \rangle$ 
7:    $c_1 \times c_2 = \langle E(0), E(a_1) \times E(b_1) \bmod m, ((E(a_1) \times E(b_2) + (E(a_2) \times E(b_1))) \bmod m), E(a_2) + E(b_2) \bmod m \rangle$ 
8:   result =  $\langle E(r_1), E(r_2), E(r_3), E(r_4) \rangle$ 
9: end procedure

```

$$E(a) = (a_1, a_2) \quad , \quad E(b) = (b_1, b_2)$$

$$E(a) \times E(b) = (0, (a_1 \times b_1) \bmod m,$$

$$(a_1 \times b_2 + a_2 \times b_1) \bmod m, (a_1 \times b_2) \bmod m)$$

(See Algorithm 3.)

- **DF.Div($E(a), E(b), m$):** The division operation cannot be carried directly as polynomials are a ring. However, in our model we delegate the division operation $E(a)/E(b)$ to be handled in rational format by the private server. Similarly, any Comparison operations that are required can be performed in the private server.

We demonstrate an unrealistic but self explainable numerical example to clarify computations with Domingo-Ferrer's homomorphic scheme below.

We choose public and private parameters as follows.

- Public parameters $(m, v) \Rightarrow (28, 2)$:
- Private parameters $(\hat{m}, \hat{r}) \Rightarrow (7, 3)$. We make sure $r^{-1} \bmod m$ exists, which is $r^{-1} = 19$ in this case.

An example formula to be calculated has two addition and a multiplication operations, $(\mathbf{x}_1 + \mathbf{x}_2) \times \mathbf{x}_3$ where $\mathbf{x}_1 = 2$, $\mathbf{x}_2 = 1$ and $\mathbf{x}_3 = 0$.

First, we transform the plaintext values $\mathbf{x}_1, \mathbf{x}_2$ and $\mathbf{x}_3$ by using Equation (1) (**transformation stage**) at a classified level.

$$x_1 = 2 \longrightarrow (0, 9)$$

$$x_2 = 1 \longrightarrow (1, 7)$$

$$x_3 = 0 \longrightarrow (2, 5)$$

Next, we encrypt the transformed plaintext values by using Equation (2) (**encryption stage**) at a classified level.

$$E(x_1) = ((0 \times 3) \bmod 28, (9 \times 9) \bmod 28) = (0, 25)$$

$$E(x_2) = ((1 \times 3) \bmod 28, (7 \times 9) \bmod 28) = (3, 7)$$

$$E(x_3) = ((2 \times 3) \bmod 28, (5 \times 9) \bmod 28) = (6, 17)$$

After that, we execute the formula $(\mathbf{x}_1 + \mathbf{x}_2) \times \mathbf{x}_3$ on ciphertext data at an unclassified level; in our case, public cloud computing. We perform the first part which is an addition operation, $(\mathbf{x}_1 + \mathbf{x}_2)$ in the encrypted domain, as follows:

$$\sum_{i=1}^2 E(x_i) = (0 + 3 \bmod 28, 25 + 7 \bmod 28) = (3, 4)$$

Then we multiply the output of the previous equations $(3, 4)$ by $E(x_3)$,

$$\begin{aligned} (3, 4) \times E(x_3) &= (3, 4) \times (6, 17) \\ &= (0, 3 \times 6 \bmod 28, (3 \times 17 + 4 \times 6) \bmod 28, 4 \times 17 \bmod 28) \\ &= (0, 18, 19, 12) \end{aligned}$$

The final ciphertext output is decrypted at a classified level based on Equation (3), and we retrieve the plaintext result as follows.

$$\begin{aligned} (0 \times 19 \bmod 28, 18 \times 19^2 \bmod 28, 19 \times 19^3 \bmod 28, 12 \times 19^4 \bmod 28) \\ = (0, 2, 9, 24) \end{aligned}$$

Finally, the decryption operation is finished by adding all elements in the vector over $Z\hat{m}$ to get $(35 \bmod 7) = 0$, which is the corresponding plaintext result if the formula is executed on plaintext domain.

3.2. Data clustering and anomaly detection

The specific anomaly detection technique that is used in the proposed model is the granular anomaly evaluation approach presented in [9]. This is primarily based on Fuzzy c-means clustering (FCM) that is adapted in a distributed manner. In the current context it is implemented over a virtual machine hierarchy in the public data centre. This hierarchy is determined in order to reflect the original hierarchical network topology that is commonly encountered in organised sensor network applications. Therefore, each such virtual machine will perform fuzzy c-means clustering in parallel for specific sensor node data as the first step. The VMs at intermediate hierarchical levels then perform an additional level of data merging in determining anomalies that are relative to that level of network granularity. Therefore, VMs at this level have access to the clustering output in the preceding levels to determine anomalies at the upper level. Therefore the anomalies identified at a certain level consist of correlation-based analysis of all data coming under that node (i.e. analysis on all leaf nodes coming under that node in the hierarchy). The adapted fuzzy c-means algorithm can be summarised as follows. First, cluster centres are randomly initiated to a user given number of expected clusters. Memberships are then calculated to each data point corresponding to each cluster centre on the basis of distance between that cluster and the data point. The closer a data point is to a cluster centre, the higher its membership towards that particular cluster centre. The algorithm stops iterating when the difference between membership values in two successive iterations is smaller than the defined termination parameter. We illustrate the main FCM clustering steps as follows.

1. Let n be the number of data points, $(p_1, p_2, \dots, p_n)$, in a specific data set D to be analysed. The algorithm randomly selects a set of clusters c .
2. Calculate the fuzzy membership μ_{ij} as follows:

$$\mu_{ij} = \frac{1}{\sum_{k=1}^c \left(\frac{d_{ij}}{d_{ik}}\right)^{2/m-1}} \quad (6)$$

where μ_{ij} is the membership of the i th data point to the j th cluster centre and d_{ij} is the Euclidean distance between i th data point and j th cluster centre. k is the iteration step and m is the fuzziness index that is defined to be 2 in the current context.

3. Update cluster centre c_{ij} as follows:

$$c_{ij} = \frac{\sum_{i=1}^n (\mu_{ij})^m p_i}{\sum_{i=1}^n (\mu_{ij})^m} \quad (7)$$

where C_{ij} is the j th cluster.

4. The fuzzy membership value of data points and cluster centres are iteratively updated in Steps 1 and 2 until the following condition is satisfied:

$$||U^{k+1} - U^k|| \leq \beta \quad (8)$$

where $U [(\mu)_{n \times c}]$ is the fuzzy membership matrix and β is the value for the termination criterion. Algorithms 4 and 5 illustrate the FCM clustering algorithm and anomaly detection process respectively.

3.3. System workflow

In this section, we explain the workflow directions between different parties to achieve secure data analysis. The model's life cycle starts when the private server receives data from any data source such as sensors. The data is encrypted by using a convenient HE algorithm and passed to collaborative public servers within the cloud computing environment to perform data clustering computations. At some point during those computations, the collaborative public servers return the encrypted results to the private server to complete the rest of computations for the first iteration. After that, the new parameters are set up and encrypted before they are passed again to collaborative public servers for the next iteration. This cycle continues until the desired results are achieved.

During the system's life cycle, the original data is encrypted only once for use by both the privacy manager (private server) and collaborative public servers during analysis.

The main entities of our cloud-based model are shown in Fig. 4. The privacy manager is responsible for three main functions: to manage the remote users accessibility within the public cloud infrastructure; send analysis tasks to collaborative public servers, and execute encryption/decryption mechanisms. The first function includes interaction with the users who are benefiting from the system's services; users communicate with the privacy manager to submit their analysis requests. Authentication and authorisation mechanisms are applied as a first defence layer of our cloud-based model. The

Algorithm 4 Anomaly detection based on FCM clustering algorithm.

```

1: procedure (1)(FCM clustering algorithm)
2:   Inputs: Data set  $D$ , Set of clusters  $c_1, c_2, \dots, c_m$ ,  $\beta$ 
3:   Outputs: Set of clusters  $c_1, c_2, \dots, c_m$ , membership matrix  $[U = u_{ij}, (u_{ij} > 0, 1 < i \leq n, 1 < j \leq m)]$ 
4:   Initialisation: Select a set of clusters  $c_1, c_2, \dots, c_m$ 
5:
6:   while  $\|U^{k+1} - U^k\| > \beta$  do
7:     \Compute a distance between each data point and each cluster centre
8:     for  $(i = 0 \text{ to } D.length)$  do
9:       for  $(j = 0 \text{ to } c.length)$  do
10:         $d_{ij} = \text{computeDist}(p_i, c_j);$  ▷ Distance computations
11:      end
12:    end
13:    \Compute a membership value between each data point and each cluster centre
14:    for  $(i = 0 \text{ to } D.length)$  do
15:      for  $(j = 0 \text{ to } c.length)$  do
16:         $u_{ij} = \text{computeMembership}(p_i, c_j);$  ▷ Membership computations
17:      end
18:    end
19:    \Calculate updated cluster centres
20:    for  $(j = 0 \text{ to } c.length)$  do
21:       $c_j = \text{computeMembership}(D, U^k);$  ▷ Update cluster centres
22:    end
23:  end
24: end procedure

```

Algorithm 5 Anomaly detection based on FCM clustering algorithm.

```

1: procedure (1)(Anomaly detection process)
2:   Inputs: Set of clusters with their data points  $[c_1[p_i, \dots], \dots, c_m[p_i, \dots]]$ ,
3:   anomaly[ ], index = 0
4:   for  $(j = 0 \text{ to } c.length)$  do
5:      $\text{mean}_j = \text{computeMean}(c_j[p_i, \dots]);$  ▷ Compute a cluster mean
6:      $\text{standardDev}_j = \text{computeStandardDev}(c_j[p_i, \dots]);$  ▷ Compute a cluster standard deviation
7:      $Tvalue_j = \text{mean}_j - \text{standardDev}_j;$ 
8:     end
9:     for  $(j = 0 \text{ to } c.length)$  do
10:      for  $(i = 0 \text{ to } c_j.length)$  do
11:        if  $p_{ij} < Tvalue_j$  then
12:          anomaly[index] =  $p_{ij};$ 
13:          index++;
14:        end
15:      end
16:    end
17:  end
18: end procedure

```

second function includes distribution of analysis tasks between collaborative public servers. The third function is related to the cryptography technique that is performed on data before analysis to ensure its privacy in the public cloud computing environment. Fig. 5 shows the workflow of the model architecture among different parties.

3.4. Security analysis

In this section we analyse the security aspects of our proposed framework. We focus on both data privacy and integrity, and also on communication security among authorised parties in the architecture.

As mentioned earlier, the privacy preservation of data relies on Domingo-Ferrer's scheme, which is based on homomorphism properties to protect data privacy. We show the parameters' settings for both encryption and decryption processes. The encryption process involves four parameters: two are public and the other are private. The public parameters are d and m : the former represents the number of splitted parts of a plaintext and the latter is a large integer ($\approx 100^{200}$) or larger. The private parameters are $r \in Z_m$ and m' which is a small divisor > 1 such as $s := \log_{m'} m$. Table 2 shows the influence of encryption/decryption parameters choices on the probability of key guessing.

The variable n is the number of random revealed pairs of plaintext-ciphertext by a malicious party. The value of s should be arbitrarily small to ensure the security of encryption/decryption parameters even if n pairs of plaintext-ciphertext are leaked. Moreover, the behaviour of encryption keys should be taken into consideration: in Domingo-Ferrer's scheme, the probability of any two keys decrypting a random ciphertext to the same plaintext is $O((\log m)/m)$. Hence, the parameter m should be increased to make this probability arbitrarily small. The security of Domingo-Ferrer's scheme relies on the fact that the subset keys consistent n revealed pairs is kept large, and the probability of any two keys yielding different plaintexts from the same ciphertext is high.

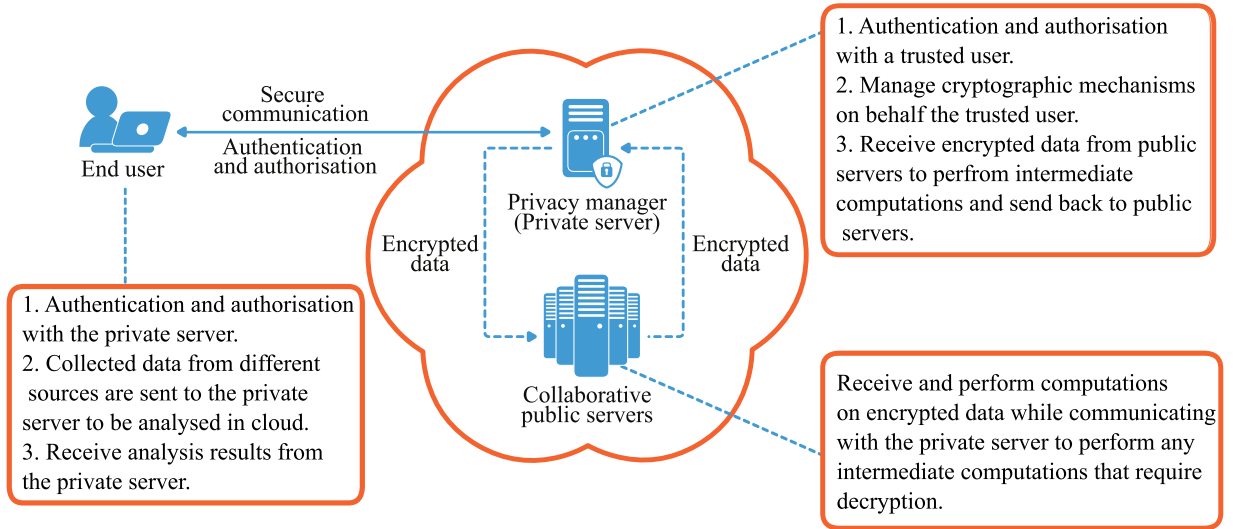


Fig. 4. The privacy-preserving distributed anomaly detection life cycle and roles of each party to achieve secure data analysis.

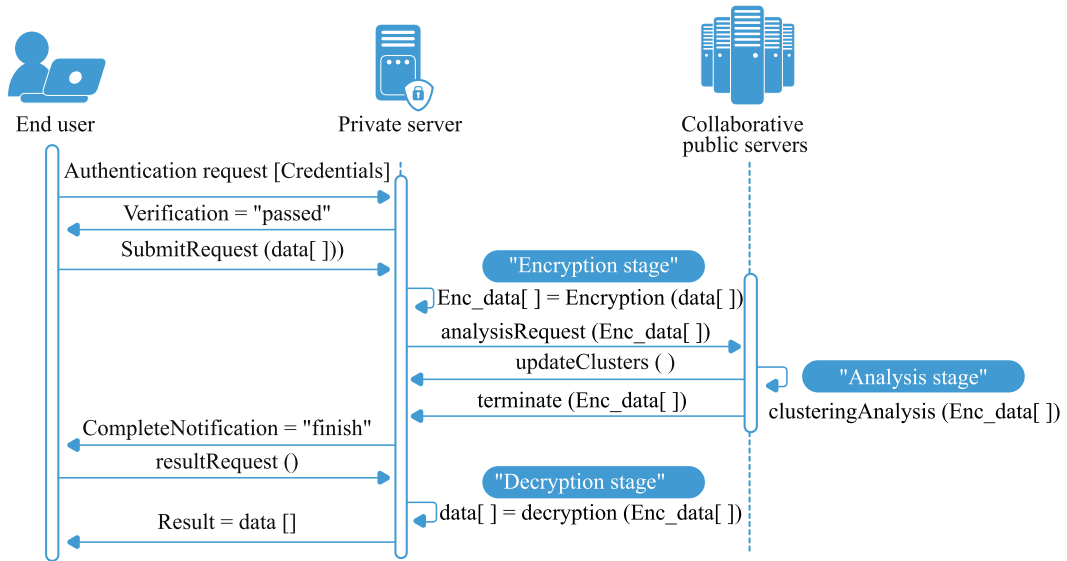


Fig. 5. A sequence diagram of the different processes in the proposed anomaly detection framework among the different entities within a cloud computing environment.

Table 2

Examples of encryption/decryption parameter choices and their relative strengths ($l(x) = \lceil \log_{10} x \rceil$).

n	s	$l(m')$	(m')	Probability of key guessing
5	6	20	120	$\approx 1.64 \times 10^{-20}$
10	11	20	220	$\approx 1.64 \times 10^{-20}$
50	51	5	255	$\approx 1.64 \times 10^{-5}$
50	53	5	265	$\approx 1.64 \times 10^{-15}$

On the other hand, Secure Socket Layer (SSL) [33] protocol is adapted to secure communication channels between different parties exchanging information. This secure communication is implemented for both outside communications, that is between data owners and cloud computing facilities, and inside communications within a cloud data centre that is, between collaborative clouds and the private server during the data analysis phase.

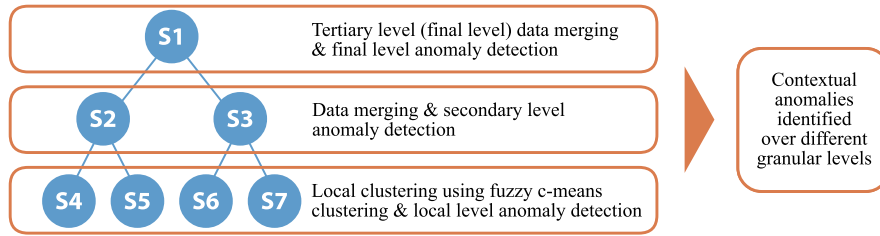


Fig. 6. Distributed anomaly detection architecture based on a hierarchy of data-processing nodes (collaborative virtual machines) that enable the identification of anomalies at different granular levels (ref: Kumarage et al. [9]).

Table 3

The data format of Intel data set.

Attribute 1	Attribute 2	Attribute 3	Attribute 4	Parameter 1	Parameter 2	Parameter 3	Parameter 4
Date	Time	Epoch	Moteid	Temperature	Humidity	Light	Voltage

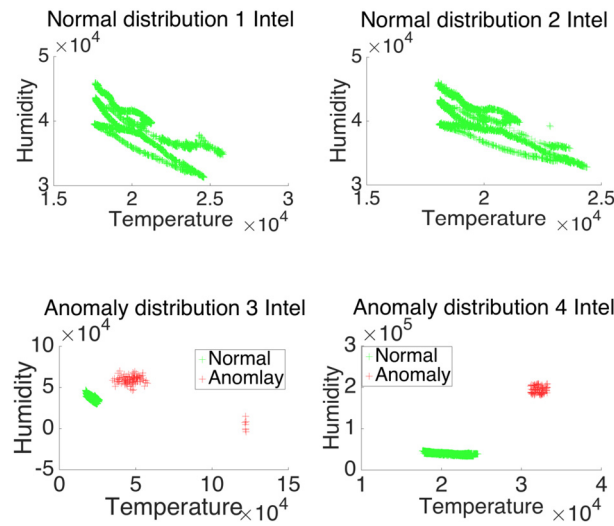


Fig. 7. The different data distributions with anomalies (bottom row) and without anomalies (top row) as derived from the Intel Berkeley Lab wireless sensor network data set.

4. Experiments and evaluation

This section demonstrates the viability of the proposed models through detailed experiments that evaluate anomaly detection accuracy and performance efficiency. Experiments are performed on the Google Cloud Platform where each data processing node in the employed hierarchical structure (see Fig. 6) of virtual machines has two Intel Xeon E5 processors, with four cores at 2.6 GHz and 12 GB of memory.

The proposed models are developed in Matlab and implemented on the Google cloud environment. Four data sets derived from the publicly-available sensor data from Intel Lab are used for evaluation. This data consists of real sensor network measurements collected from 54 sensors deployed at the Intel Berkeley Research Laboratory [34] and include time-stamped information on temperature, humidity, light and voltage values. The format of the data set is presented in Table 3. Each derived data set consists of 2800 data points with distributions 1 and 2 having zero anomalies and distributions 3 and 4 having 100 anomalies each. Fig. 7, shows the distribution of derived Intel data sets based on the two attributes of humidity and temperature. For each data set, we ran six levels of experiments corresponding to the expected number of clusters from 7 to 12 in the proposed data clustering based anomaly detection approach. This range is selected as any number below that will be too small to optimally represent a general sensor data distribution to score comparable fuzzy values, while a larger number would increase the computational complexity beyond what is needed for the task of accurate anomaly detection [9].

4.1. Methodology and evaluation criteria

In our implementation, the specific cluster analysis algorithms underlying the distributed anomaly detection approach are based on the work proposed in [9]. We derive our models using this as a foundation to perform anomaly detection in

Table 4

Classification accuracy for INTEL normal distribution 1 and 2 at nodes S4 and S5 (Plaintext version – (P)) and (Ciphertext version – (C)).

No. clusters	FN(P)	FN(C)	FP(P)	FP(C)	Sen(P)	Sen(C)	Spec(P)	Spec(C)
7	0	0	0	0	NAN	NAN	1	1
8	0	0	0	0	NAN	NAN	1	1
9	0	0	0	0	NAN	NAN	1	1
10	0	0	0	0	NAN	NAN	1	1
11	0	0	0	0	NAN	NAN	1	1
12	0	0	0	0	NAN	NAN	1	1

Table 5

Classification accuracy for INTEL anomaly distribution 3 at node S6 (Plaintext version – (P)) and (Ciphertext version – (C)).

No. clusters	FN(P)	FN(C)	FP(P)	FP(C)	Sen(P)	Sen(C)	Spec(P)	Spec(C)
7	15	13	2	3	0.8598	0.8785	0.9996	0.9993
8	9	10	11	6	0.9159	0.9065	0.9976	0.9986
9	9	7	25	8	0.9159	0.9345	0.9946	0.9985
10	8	7	2	9	0.9252	0.9345	0.9996	0.9980
11	8	9	2	2	0.9252	0.9158	0.9996	0.9995
12	12	11	5	16	0.8879	0.8971	0.9989	0.9965

a secure, privacy-preserving manner with a unique lightweight homomorphic approach for data processing. Therefore, the accuracy of the proposed distributed anomaly detection model using fuzzy c-means clustering (FCM) [35] is evaluated in a manner so that output accuracy is directly comparable with the accuracy findings in the above non-homomorphic plaintext anomaly detection model which is the inspiration for the current work. The current results are based on experiments that uses a subset of the data in [9] and the output results of accuracy analysis for anomaly detection over the data processing hierarchy can directly be compared to the plaintext version accuracy as presented in [9]. We show that the proposed framework successfully achieves the same levels of accuracy while performing operations in cloud using a virtual machine hierarchy with integrated lightweight homomorphic encryption.

For each data set, the statistical measures of sensitivity and specificity are calculated through four values, namely, the number of, True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN) over the different levels in the anomaly detection process. The sensitivity is the probability of a statistical test to be positive based on an identified TP value, while the specificity is the probability of a statistical test to be negative based on an identified TN value.

$$\text{Sensitivity} = \frac{\text{Number of True Positive values}}{(\text{Number of True Positive values} + \text{Number of False Negative values})}$$

$$\text{Specificity} = \frac{\text{Number of True Negative values}}{(\text{Number of True Negative values} + \text{Number of False Negative values})}$$

The sensitivity and specificity measurements are calculated in three levels of the granular data clustering model with the Intel Lab data sets distributed over the four local level clustering nodes (S4–S7) in the VM hierarchy. Next, the second iteration of data clustering occurs in the secondary level of processing nodes S2 and S3 based on extracted results from the previous level. The final clustering and anomaly detection results are computed in the final level at root node S1.

4.2. Anomaly detection accuracy

We implemented two versions of the proposed framework, one that processes plaintext data and the other processing ciphertext data through the integration of lightweight homomorphic encryption. The results are presented comparatively for the four data sets (1–4 INTEL Data Distributions – Fig. 7) over the three different levels in the data processing hierarchy as shown in Fig. 6. The individual data distributions from 1–4 are taken to be processed at leaf nodes S4–S6. Tables 4–9 show the cluster analysis based results for anomaly detection accuracy in both versions. We indicate the plaintext version with postfix (P) and the ciphertext version with postfix (C). We refer (Sen) and (Spec) to the sensitivity and the specificity respectively.

Observing the results in Table 4 through to Table 9, we can infer that both plaintext and ciphertext versions have achieved high levels of detection accuracy as measured by the sensitivity and specificity metrics at all nodes in the hierarchical structure. Table 4, shows that ideal detection accuracy is achieved with 0 false positive rate for non-anomalous data encountered in leaf nodes S4 and S5 in both plaintext and ciphertext versions. Tables 5 and 6 show that the sensitivity

Table 6

Classification accuracy for INTEL anomaly distribution 4 at node S7 (Plaintext version – (P)) and (Ciphertext version – (C)).

No. clusters	FN(P)	FN(C)	FP(P)	FP(C)	Sen(P)	Sen(C)	Spec(P)	Spec(C)
7	0	0	196	58	1	1	0.9574	0.9873
8	0	0	1	7	1	1	0.9998	0.9984
9	0	0	1	5	1	1	0.9998	0.9989
10	0	0	32	13	1	1	0.9930	0.9971
11	0	0	32	9	1	1	0.99930	0.9980
12	0	0	1	19	1	1	0.9998	0.9958

Table 7

Classification accuracy for INTEL normal distributions 1 and 2 at parent node S2 (Plaintext version – (P)) and (Ciphertext version – (C)).

No. clusters	FN(P)	FN(C)	FP(P)	FP(C)	Sen(P)	Sen(C)	Spec(P)	Spec(C)
7	0	0	0	0	NAN	NAN	1	1
8	0	0	0	5	NAN	NAN	1	0.9994
9	0	0	32	15	NAN	NAN	0.9965	0.9983
10	0	0	19	22	NAN	NAN	0.9979	0.9976
11	0	0	76	102	NAN	NAN	0.9918	0.9890
12	0	0	11	27	NAN	NAN	0.9988	0.9970

Table 8

Classification accuracy for INTEL anomaly distributions 3 and 4 at parent node S3 (Plaintext version – (P)) and (Ciphertext version – (C)).

No. clusters	FN(P)	FN(C)	FP(P)	FP(C)	Sen(P)	Sen(C)	Spec(P)	Spec(C)
7	3	0	32	15	0.9855	1	0.9965	0.9983
8	0	5	19	6	1	0.9758	0.9979	0.9993
9	0	0	76	41	1	1	0.9917	0.9955
10	18	10	22	7	0.9656	0.9516	0.9976	0.9992
11	25	23	5	31	0.8797	0.8889	0.9994	0.9966
12	14	12	39	62	0.9323	0.9420	0.9957	0.9932

average achieved for the two data sets with anomalies are 0.9049 and 1.0000 for plaintext while it is 0.9111 and 1.0000 for ciphertext over the cluster range from 7 to 12 with regard to local level anomaly detection at leaf nodes S6 and S7 in the data processing hierarchy.

Accuracy results at the secondary level of anomaly detection are given in [Tables 7 and 8](#). Parent nodes S2 and S3 achieves high detection accuracy with a specificity average of 0.9957 in plaintext and 0.9964 in ciphertext observed for the two merged distributions without any anomalies at node S2. The corresponding average specificity values for S3 are at 0.9968 in plaintext and 0.9970 in ciphertext with the number of clusters varying from 7 to 12 for the two merged distributions with anomalies present. The average sensitivity observed at this node is 0.9605 for plaintext and 0.9597 for ciphertext.

[Table 9](#), gives the result for the final level of anomaly detection performed at the root node of the hierarchy (S1). The average sensitivity and specificity for plaintext remains high at 0.9476 and 0.9972, while the average sensitivity and specificity for ciphertext are at 0.9492 and 0.9972. Therefore, the overall results clearly demonstrate that the proposed ciphertext version of clustering based anomaly detection can achieve significantly high accuracy results as to be comparable with the corresponding plaintext values that are observed as well as when compared to the results obtained over the same data sets for the work presented in [\[9\]](#). This is while providing the added advantage of privacy-preserving in a cloud environment using a lightweight homomorphic encryption approach.

[Fig. 8](#), visually shows the distribution of identified clusters and detected anomalies for Intel normal distribution 1 and the distribution with anomalies 3. The first row depicts the clustering for normal distribution 1 while the second row depicts the clustering for anomaly distribution 3 with the third row showing the isolated anomalies for this distribution. In [Fig. 9](#), the first row depicts the clustering output for Intel normal distribution 2 while the second and third rows consecutively show the clustering and isolated anomalies for Intel anomaly distribution 4. Results of the final clustering stage are shown in [Table 9](#).

4.3. Performance efficiency

A key achievement in the proposed framework is that it performs privacy preserving anomaly detection in a lightweight fashion as opposed to more computationally intensive methods that rely on HE. We examine performance efficiency in terms of the execution time for the ciphertext cluster analysis and anomaly detection process for different levels in the

Table 9

Final classification accuracy for INTEL anomaly distributions 1, 3 and normal distributions 2, 4 at root node S1 (Plaintext version – (P)) and (Ciphertext version – (C)).

No. clusters	FN(P)	FN(C)	FP(P)	FP(C)	Sen(P)	Sen(C)	Spec(P)	Spec(C)
7	15	13	198	172	0.9275	0.9371	0.9894	0.9894
8	9	10	12	14	0.9565	0.9516	0.9994	0.9992
9	9	9	26	23	0.9565	0.9565	0.9986	0.9987
10	8	9	34	35	0.9614	0.9565	0.9982	0.9981
11	8	8	34	31	0.9420	0.9613	0.9982	0.9983
12	12	14	6	9	0.9420	0.9323	0.9997	0.9995

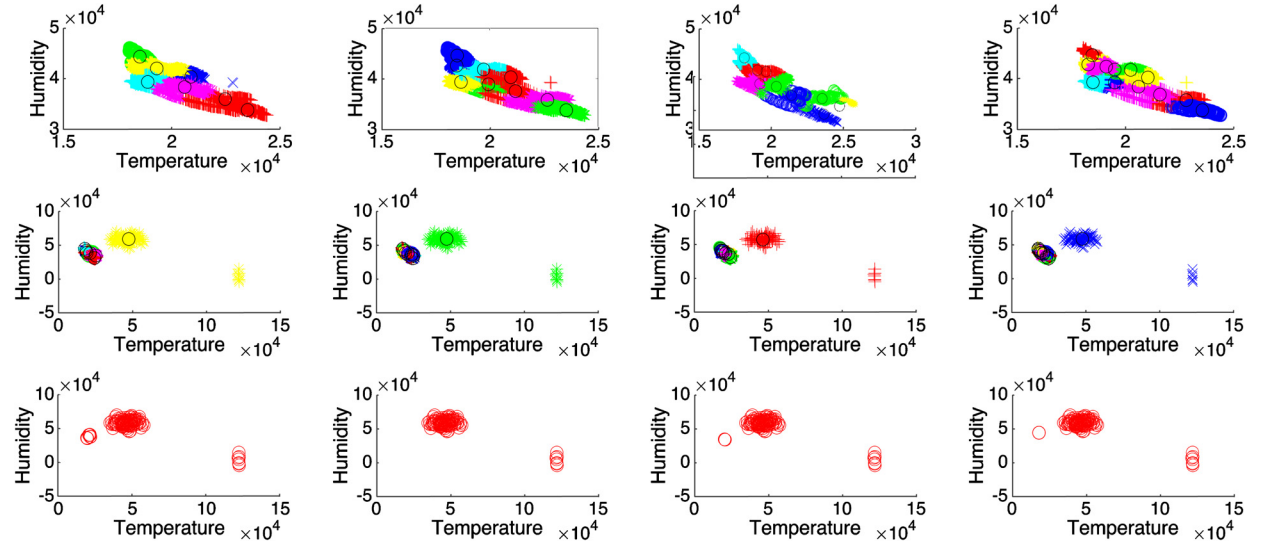


Fig. 8. Data clustering output (from left to right) for 7, 9, 10 and 12 expected clusters for INTEL normal distribution 1 (row 1) and anomaly distribution 3 (row 2) with the detected anomalies for distribution 3 (row 3) using the proposed model to perform cloud based distributed anomaly detection in a privacy preserving manner.

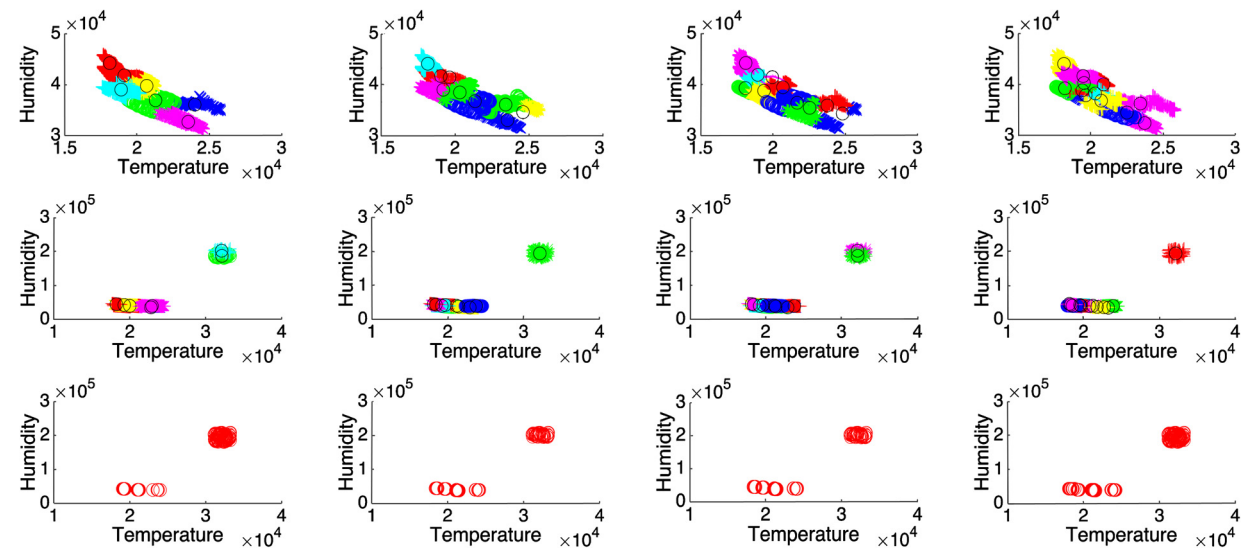


Fig. 9. Data clustering output (from left to right) for 7, 9, 10 and 12 expected clusters for INTEL normal distribution 2 (row 1) and anomaly distribution 4 (row 2) with the detected anomalies for distribution 3 (row 3) using the proposed model to perform cloud based distributed anomaly detection in a privacy preserving manner.

Table 10

The average execution time of encryption and decryption processes for different amounts of data (unit: millisecond).

Process type	# of data points		
	1200	2400	4700
Encryption	11.9418	23.7261	46.2950
Decryption	5.0038	11.0750	23.594

Table 11

The average execution time for ciphertext cluster analysis and anomaly detection at the local clustering level (leaf nodes) with varying data set size (unit: millisecond).

No. clusters	# of data points		
	1200	2400	4700
7	259.1736	504.0812	1096.5587
8	281.9739	552.1109	1152.9083
9	301.3022	620.3400	1240.2602
10	359.4110	693.0236	1309.2317
11	412.1381	705.6102	1372.3950
12	473.4015	740.9021	1425.1055

Table 12

The average execution time of ciphertext data clustering and anomaly detection for the complete process including all three levels of the hierarchical data processing structure (unit: millisecond).

No. clusters	# of data points		
	1200	2400	4700
7	529.5813	764.5448	1326.3192
8	547.9503	809.6010	1412.0183
9	579.8740	875.2913	1487.2622
10	639.6551	839.2039	1540.2317
11	685.1209	941.6291	1604.3212
12	730.2901	1015.0047	1673.1655

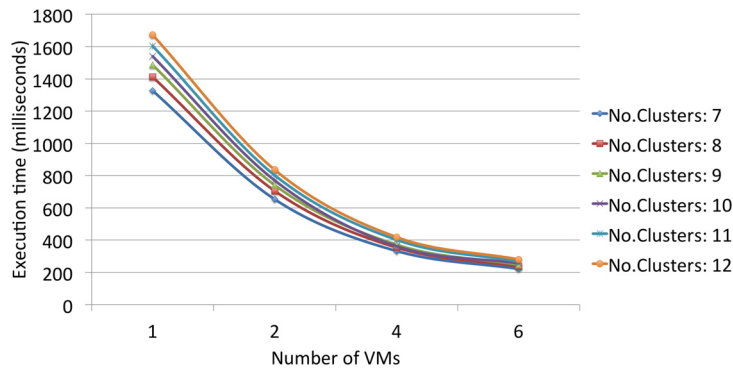


Fig. 10. The variation of average execution time for the proposed model using Intel normal distribution 1 at the local clustering level with varying numbers of VMs and clusters.

hierarchical structure as in [9]. The average execution times for encryption and decryption processes over different amounts of data are given in Table 10. The variation in average execution time for data clustering and anomaly detection tasks at the local level in the hierarchy (where data processing is most intensive) when the amounts of data increase are given in Table 11. This is divided into 3 sets of data points over which the clusters are formed for anomaly detection with the number of clusters varying from 7 to 12. Table 12 shows the average execution time for ciphertext data clustering and anomaly detection considering all three levels (complete process from leaf node to root) of the proposed hierarchical data processing framework.

To overcome the increase in execution time, the proposed model distributes the cluster formation and subsequent anomaly detection tasks at each local node among a larger sets of VMs. In general, increasing the number of VMs at each data processing level leads to a rapid decrease in the execution time while managing computational complexity as desired. Fig. 10 illustrates how the increase in the number of VMs for data processing decreases the execution time over different numbers of expected clusters.

Table 13

The average number of iterations (with each iteration signifying one bidirectional data communication cycle between the private server and a public server or VM) that are required based on different number of clusters as the amount of data varies.

No. clusters	# of data points		
	1200	2400	4700
7	35	44	76
8	40	56	82
9	57	61	85
10	66	74	93
11	78	92	101
12	95	105	112

The communication overhead imposed by the use of a private server that collaborates with other public cloud computing infrastructure in the proposed model is examined in Table 13. This gives the communication overhead (number of data points communicated) based on different cluster settings and different amounts of processed data.

Each iteration involves data exchange between public cloud VMs and the private server. Most operations are performed on the data that are stored as ciphertext values in the public cloud. However, some intermediate operations cannot be performed on the public cloud without decryption due to limitations in the adopted homomorphic scheme. The amount of data that is required to be transferred between public cloud servers and the private server can be expressed as follows.

Each data point (plaintext value) can be considered as n bits. The encrypted data point (ciphertext value) is based on parameter d in a ciphertext algorithm setting. If we assume $d = 2$, then the ciphertext value is $n * 2$. If we have i ciphertext values, then the amount of data that is required to be sent in each iteration is $(i * (n * 2))$. These values can vary based on different clustering and encryption/decryption algorithm settings.

4.4. Execution time comparison

Table 14 provides a detailed comparison on ciphertext execution time of the proposed model with that of the ciphertext execution time of Craig Gentry's Homomorphic scheme if adapted over the proposed data clustering and anomaly detection framework. Each of the data processing tasks is analysed considering the different sub-tasks under encryption/decryption, cluster formation and anomaly detection over the local, intermediate and final levels of a three level data processing hierarchy as relative to the proposed model. The data is presented per node basis with n representing the number of data points, i the number of iterations before converging and c the number of clusters. The average execution time per operation is given in seconds with input directly from performed experiments for the proposed model and from [36] for Craig Gentry's Homomorphic scheme. The machine specifications for experiments in both schemes are dual processor systems with Intel Xeon E5 processors having four cores each. Our model employs 12 Gb of RAM with processor speeds of 2.6 GHz while Craig Gentry's scheme employs 16 Gb of RAM with processor speeds at 3.0 GHz. Therefore, the machine specifications for Craig Gentry's model implementation over which the following execution times are calculated remain high in comparison to the machine specifications where our model is implemented. However, even with the added advantage it is clearly shown that the proposed model has a significantly lighter footprint as compared through execution time. The operational breakdown presented in Table 14 clearly demonstrates that by adapting Domingo Ferrer's scheme for homomorphic encryption results in significant reduction of computational complexity as evidenced by the execution times that are significantly less than if Craig Gentry's Homomorphic scheme is adapted over the same framework.

Therefore, adoption of Domingo-Ferrer's scheme in the proposed model enables lightweight arithmetic computations to be performed in a far efficient manner compared to other HE schemes. This feature as demonstrated in the above table forms the core strength of the proposed model in terms of providing scalable and privacy preserving anomaly detection for large-scale data with significantly less computational complexity while retaining high accuracy in anomaly detection.

5. Conclusion

In this paper, we proposed a unique cloud-based framework that is lightweight and has the ability to perform scalable anomaly detection in a secure and privacy-preserving manner. An inherently practical anomaly detection scheme based on distributed fuzzy c-means (FCM) clustering was integrated with an efficient lightweight cryptography technique in the form of Domingo-Ferrer's additive and multiplicative privacy homomorphism to perform operations on encrypted data using a cloud based collaborative model. A private server acting as a privacy manager collaborates with a set of public servers to perform the required tasks efficiently within the capabilities of the adopted homomorphic approach. The proposed framework can be adapted efficiently in a scalable manner for critical anomaly detection applications that wish to take advantage of cloud computing infrastructure without compromising data security and privacy. The experiments described in this paper have demonstrated the capability of proposed models in accurate detection of data anomalies without any compromise in data privacy or any associated overheads in computational and communication complexity. Future research will improve

Table 14

The average execution time of involved operations for Domingo-Ferrer's and Craig Gentry's homomorphic schemes (i is the number of clustering iterations, with n the number of data points and c the number of clusters).

Local clustering & anomaly detection level (per node)		
Number of operations	Domingo-Ferrer's scheme	Craig Gentry's scheme
Encryption phase		
Encryption: $Enc = n$	Encryption: $Enc \times 0.00985$ (per value: 7 bits) ----- In case $n = 4700$, Execution time ≈ 0.77 seconds $\times i$	Encryption: $Enc \times 1450$ (per bit) ----- In case $n = 4700$, Execution time ≈ 220 hours $\times i$
Data clustering phase		
DistanceCalculation: $D = (n \times c \times 5)$ MeanCalculation: $M = n$ Updateclusters: $U = (n^3 + n) \times c$	Distance: $D \times i$ {Add ≈ 0.00985 , Sub ≈ 0.00985 , Mult ≈ 0.00727 } (per value) Mean: $M \times i$ {Add ≈ 0.00985 } (per value) Updateclusters: $S \times i$ {Add ≈ 0.00985 , Mult ≈ 0.00727 } (per value) ----- In case $n = 4700$, Execution time ≈ 46.8 minutes $\times i$	Distance: $D \times i$ {Add & Sub $< 1 \mu s$, Mult $\approx 1.76s$ } (per bit) Mean: $M \times i$ {Add $< 1 \mu s$ } (per bit) Updateclusters: $S \times i$ {Add $< 1 \mu s$, Mult $\approx 1.76s$ } (per bit) ----- In case $n = 4700$, Execution time ≈ 1260 hours $\times i$
Anomaly detection phase		
Meancalculation: $M = n$ Threshold(Standarddeviation) application: $S = (n^2 + n)$	Mean: $M \times i$ Threshold(Standarddeviation) application: $S \times i$ ----- In case $n = 4700$, Execution time ≈ 2.8 seconds $\times i$	Mean: $M \times i$ {Add $< 1 \mu s$ } Threshold(Standarddeviation) application: $S \times i$ ----- In case $n = 4700$, Execution time ≈ 1.4 hours $\times i$
Secondary anomaly detection level (per node)		
Anomaly detection phase		
Mean: $M = n$ Threshold(Standarddeviation) application: $S = (n^2 + n)$	Mean: $M \times i$ Threshold(Standarddeviation) application: $S \times i$ ----- In case $n = 9400$, Execution time ≈ 5.6 seconds $\times i$	Mean: $M \times i$ Threshold(Standarddeviation) application: $S \times i$ ----- In case $n = 4700$, Execution time ≈ 2.6 hours $\times i$
Final anomaly detection level		
Anomaly detection phase		
Mean: $M = n$ Threshold(Standarddeviation) application: $S = (n^2 + n)$	Mean: $M \times i$ Threshold(Standarddeviation) application: $S \times i$ ----- In case $n = 18800$, Execution time ≈ 11.3 seconds $\times i$	Mean: $M \times i$ Threshold(Standarddeviation) application: $S \times i$ ----- In case $n = 18800$, Execution time ≈ 5.3 hours $\times i$
Decryption phase		
Decryption: $Dec = n$	Decryption: $Dec \times 0.00502$ ----- In case $n = 4700$, Execution time ≈ 0.39 seconds $\times i$	Decryption: $Dec \times 200$ ----- In case $n = 4700$, Execution time ≈ 4.3 hours $\times i$

the model in terms of further reducing communication overheads as well as enhancing the capabilities of the adapted Homomorphic Encryption technique to cover more operations without any trade-offs in efficiency.

References

- [1] C.C. Aggarwal, *Data Mining – The Textbook*, Springer, 2015.
- [2] K. Li, H. Jiang, L.T. Yang, A. Cuzzocrea (Eds.), *Big Data – Algorithms, Analytics, and Applications*, Chapman and Hall/CRC, 2015, <http://www.crcnetbase.com/isbn/978-1-4822-4056-6>.
- [3] R. Agrawal, A. Kadadi, X. Dai, F. Andrès, Challenges and opportunities with big data visualization, in: *Proceedings of the 7th International Conference on Management of Computational and Collective Intelligence in Digital EcoSystems*, Caragatatuba, Brazil, October 25–29, 2015, 2015, pp. 169–173.
- [4] I.A.T. Hashem, I. Yaqoob, N.B. Anuar, S. Mokhtar, A. Gani, S.U. Khan, The rise of “big data” on cloud computing: review and open research issues, *Inf. Syst.* 47 (2015) 98–115, <http://dx.doi.org/10.1016/j.is.2014.07.006>.

- [5] I. Gvero, Cloud computing concepts, technology and architecture by Thomas Erl, Zaigham Mahmood and Ricardo Puttini, ACM SIGSOFT Softw. Eng. Notes 39 (4) (2014) 37–38, <http://dx.doi.org/10.1145/2632434.2632462>.
- [6] S. Subashini, V. Kavitha, A survey on security issues in service delivery models of cloud computing, J. Netw. Comput. Appl. 34 (1) (2011) 1–11, <http://dx.doi.org/10.1016/j.jnca.2010.07.006>.
- [7] D. Zisis, D. Lekkas, Addressing cloud computing security issues, Future Gener. Comput. Syst. 28 (3) (2012) 583–592, <http://dx.doi.org/10.1016/j.future.2010.12.006>.
- [8] K.W. Hamlen, M. Kantarcioglu, L. Khan, B.M. Thuraisingham, Security issues for cloud computing, Int. J. Inf. Secur. Priv. 4 (2) (2010) 36–48, <http://dx.doi.org/10.4018/jisp.2010040103>.
- [9] H. Kumarage, I. Khalil, Z. Tari, A.Y. Zomaya, Distributed anomaly detection for industrial wireless sensor networks based on fuzzy data modelling, J. Parallel Distrib. Comput. 73 (6) (2013) 790–806, <http://dx.doi.org/10.1016/j.jpdc.2013.02.004>.
- [10] J. Domingo-Ferrer, A provably secure additive and multiplicative privacy homomorphism, in: Information Security, 5th International Conference, ISC 2002 Sao Paulo, Brazil, September 30–October 2, 2002, Proceedings, 2002, pp. 471–483.
- [11] S. Yakubov, V. Gadepally, N. Scheer, E. Shen, A. Yerukhimovich, A survey of cryptographic approaches to securing big-data analytics in the cloud, in: IEEE High Performance Extreme Computing Conference, HPEC 2014, Waltham, MA, USA, September 9–11, 2014, 2014, pp. 1–6.
- [12] A.C. Yao, Protocols for secure computations (extended abstract), in: 23rd Annual Symposium on Foundations of Computer Science, Chicago, Illinois, USA, 3–5 November 1982, 1982, pp. 160–164.
- [13] D. Chaum, C. Crépeau, I. Damgård, Multiparty unconditionally secure protocols (extended abstract), in: Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2–4, 1988, Chicago, Illinois, USA, 1988, pp. 11–19.
- [14] A.C. Yao, How to generate and exchange secrets (extended abstract), in: 27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27–29 October 1986, 1986, pp. 162–167.
- [15] M. Ben-Or, S. Goldwasser, A. Wigderson, Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract), in: Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2–4, 1988, Chicago, Illinois, USA, 1988, pp. 1–10.
- [16] B. Parno, J. Howell, C. Gentry, M. Raykova, Pinocchio: nearly practical verifiable computation, Commun. ACM 59 (2) (2016) 103–112, <http://dx.doi.org/10.1145/2856449>.
- [17] R.L. Rivest, A. Shamir, L.M. Adleman, A method for obtaining digital signatures and public-key cryptosystems, Commun. ACM 21 (2) (1978) 120–126, <http://dx.doi.org/10.1145/359340.359342>.
- [18] T.E. Gamal, A public key cryptosystem and a signature scheme based on discrete logarithms, IEEE Trans. Inf. Theory 31 (4) (1985) 469–472, <http://dx.doi.org/10.1109/TIT.1985.1057074>.
- [19] L. Fousse, P. Lafourcade, M. Alnuaimi, Benaloh's dense probabilistic encryption revisited, in: Progress in Cryptology – AFRICACRYPT 2011 – 4th International Conference on Cryptology in Africa, Dakar, Senegal, July 5–7, 2011, Proceedings, 2011, pp. 348–362.
- [20] P. Paillier, Public-key cryptosystems based on composite degree residuosity classes, in: Advances in Cryptology – EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2–6, 1999, Proceeding, 1999, pp. 223–238.
- [21] C. Gentry, Fully homomorphic encryption using ideal lattices, in: Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC '09, ACM, New York, NY, USA, 2009, pp. 169–178.
- [22] Z. Brakerski, V. Vaikuntanathan, Fully homomorphic encryption from ring-lwe and security for key dependent messages, in: Advances in Cryptology – CRYPTO 2011 – 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14–18, 2011, Proceedings, 2011, pp. 505–524.
- [23] J. Coron, A. Mandal, D. Naccache, M. Tibouchi, Fully homomorphic encryption over the integers with shorter public keys, in: Advances in Cryptology – CRYPTO 2011 – 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14–18, 2011, Proceedings, 2011, pp. 487–504.
- [24] J.H. Cheon, J. Coron, J. Kim, M.S. Lee, T. Lepoint, M. Tibouchi, A. Yun, Batch fully homomorphic encryption over the integers, in: Advances in Cryptology – EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26–30, 2013, Proceedings, 2013, pp. 315–335.
- [25] Z. Brakerski, C. Gentry, V. Vaikuntanathan, (Leveled) Fully homomorphic encryption without bootstrapping, ACM Trans. Comput. Theory 6 (3) (2014) 13:1–13:36, <http://dx.doi.org/10.1145/2633600>.
- [26] C. Fontaine, F. Galand, A survey of homomorphic encryption for nonspecialists, EURASIP J. Inf. Secur. (2007), <http://dx.doi.org/10.1155/2007/13801>.
- [27] T. Okamoto, S. Uchiyama, A new public-key cryptosystem as secure as factoring, in: Advances in Cryptology – EUROCRYPT '98, International Conference on the Theory and Application of Cryptographic Techniques, Espoo, Finland, May 31–June 4, 1998, Proceeding, 1998, pp. 308–318.
- [28] D. Naccache, J. Stern, A new public key cryptosystem based on higher residues, in: CCS '98, Proceedings of the 5th ACM Conference on Computer and Communications Security, San Francisco, CA, USA, November 3–5, 1998, 1998, pp. 59–66.
- [29] D. Hankerson, A.J. Menezes, S. Vanstone, Guide to Elliptic Curve Cryptography, Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2003.
- [30] J. Domingo-Ferrer, A provably secure additive and multiplicative privacy homomorphism, in: Information Security, 5th International Conference, ISC 2002 Sao Paulo, Brazil, September 30–October 2, 2002, Proceedings, 2002, pp. 471–483.
- [31] D. Wagner, Cryptanalysis of an algebraic privacy homomorphism, in: Information Security, 6th International Conference, ISC 2003, Bristol, UK, October 1–3, 2003, Proceedings, 2003, pp. 234–239.
- [32] S. Pearson, Y. Shen, M. Mowbray, A privacy manager for cloud computing, in: Cloud Computing, First International Conference, CloudCom 2009, Beijing, China, December 1–4, 2009, Proceedings, 2009, pp. 90–106.
- [33] E. Rescorla, SSL and TLS: Designing and Building Secure Systems, vol. 1, Addison-Wesley, Reading, 2001.
- [34] P. Bodik, W. Hong, C. Guestrin, S. Madden, M. Paskin, R. Thibaux, Intel lab data, db.csail.mit.edu/labdata/labdata.html, 2004.
- [35] R.L. Cannon, J.V. Dave, J.C. Bezdek, Efficient implementation of the fuzzy c-means clustering algorithms, IEEE Trans. Pattern Anal. Mach. Intell. 8 (2) (1986) 248–255, <http://dx.doi.org/10.1109/TPAMI.1986.4767778>.
- [36] Ö. Kocabaş, T. Soyata, Medical data analytics in the cloud using homomorphic encryption, in: Handbook of Research on Cloud Infrastructures for Big Data Analytics, 2014, p. 471.